Manuel complet

Stores enrouleurs — XIAO ESP32-C3 · motoréducteurs CC à encodeur module DRV8833 · v5.2

Croquis `StoreEnrouleur_MoteurCC_Encodeur.ino` + `favicon.h` — octobre 2026

câblage : module DRV8833, nappe moteur

Sommaire

1. Présentation
2. Matériel
3. Montage et câblage
4. Installation du programme
5. Première mise en service et calibration
6. Utilisation de la page web
7. Fonctionnement interne
8. Réglages du programme
9. Réseau Wi-Fi
10. Mises à jour (OTA)
11. Interface de programmation (API HTTP)
12. Messages du moniteur série
13. Dépannage
14. Sécurité, limites et entretien
15. Annexes

1. Présentation

Ce montage commande **un ou deux stores enrouleurs** motorisés depuis une **page web** hébergée par la carte elle-même. Il ne dépend d'aucun service extérieur (plus de Blynk) : un navigateur sur le réseau local suffit.

Depuis la v5, chaque store est entraîné par un **motoréducteur à courant continu 6 V équipé d'un encodeur magnétique** (GBMQ-GM12BY20, type GM12 / N20), piloté par un **module DRV8833** (pont en H double). L'encodeur informe en permanence le programme de la rotation réelle de l'arbre : la position du store est donc **mesurée** et non plus supposée, comme avec les moteurs pas à pas des versions précédentes.

Fonctions principales

Fonction	Détail
Commande	Monter, Stop, Descendre, Aller à X % — pour chaque store ou les deux ensemble
Mesure de position	Encodeur en quadrature décodé ×4 sous interruptions ; 0 = haut, « course » = bas
Pilotage moteur	PWM 20 kHz inaudible, mode « décroissance lente », rampe d'accélération, ralentissement en fin de course
Arrêt	Freinage actif (bobinage court-circuité) — le moteur résiste à la rotation sans consommer
Sécurités	Détection de blocage, détection de sens incohérent, surveillance du défaut DRV8833 (broche ULT / nFAULT), pause freinée avant inversion
Mémoire	Course et position enregistrées en mémoire non volatile (NVS), retrouvées après coupure
Réseau	Wi-Fi de la box, nom `store.local` ; point d'accès de secours en 192.168.5.1 avec retour automatique sur la box ; plusieurs cartes possibles sur le même réseau (§ 9.1)

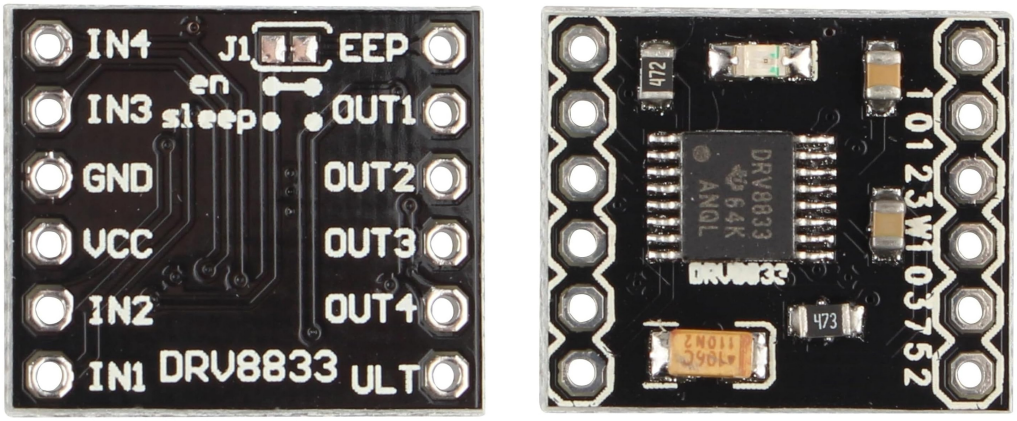
Fonction	Détail
Mises à jour	OTA depuis l'Arduino IDE ou par le navigateur (`/update`), protégées par mot de passe
Icône **(v5.2)**	Icône de la page dans l'onglet du navigateur et sur l'écran d'accueil du téléphone (fichier `favicon.h`)
Intégration	API HTTP simple (`/api/status` , `/api/cmd`) pour une domotique

2. Matériel

Élément	Référence	Remarques
Microcontrôleur	Seeed Studio XIAO ESP32-C3	Wi-Fi 2,4 GHz, USB intégré
Pont en H double	Module DRV8833 (carte noire, repères IN1–IN4, OUT1–OUT4, VCC, GND, EEP, ULT, pont J1)	VCC de 2,7 à 10,8 V ; un canal par moteur. Condensateurs de découplage, VINT et VCP **déjà soudés** sur le module
Moteurs	2 × motoréducteur GBMQ-GM12BY20, 6 V, 70 tr/min, à encodeur magnétique	Nappe 6 fils : rouge M1, blanc M2, marron C1, vert C2, bleu VCC, noir GND. Encodeur alimenté en 3,3 V
Alimentation	Bloc 5 V régulé	**1 A minimum** (plus si les moteurs forcent)
Résistance de rappel	1 × 10 kΩ	ULT vers 3,3 V, seulement si ULT n'est pas déjà tirée au 5 V par le module (voir 3.2)
Pont J1 du module	Goutte de soudure	Met la puce en marche en permanence (EEP relié à VCC)

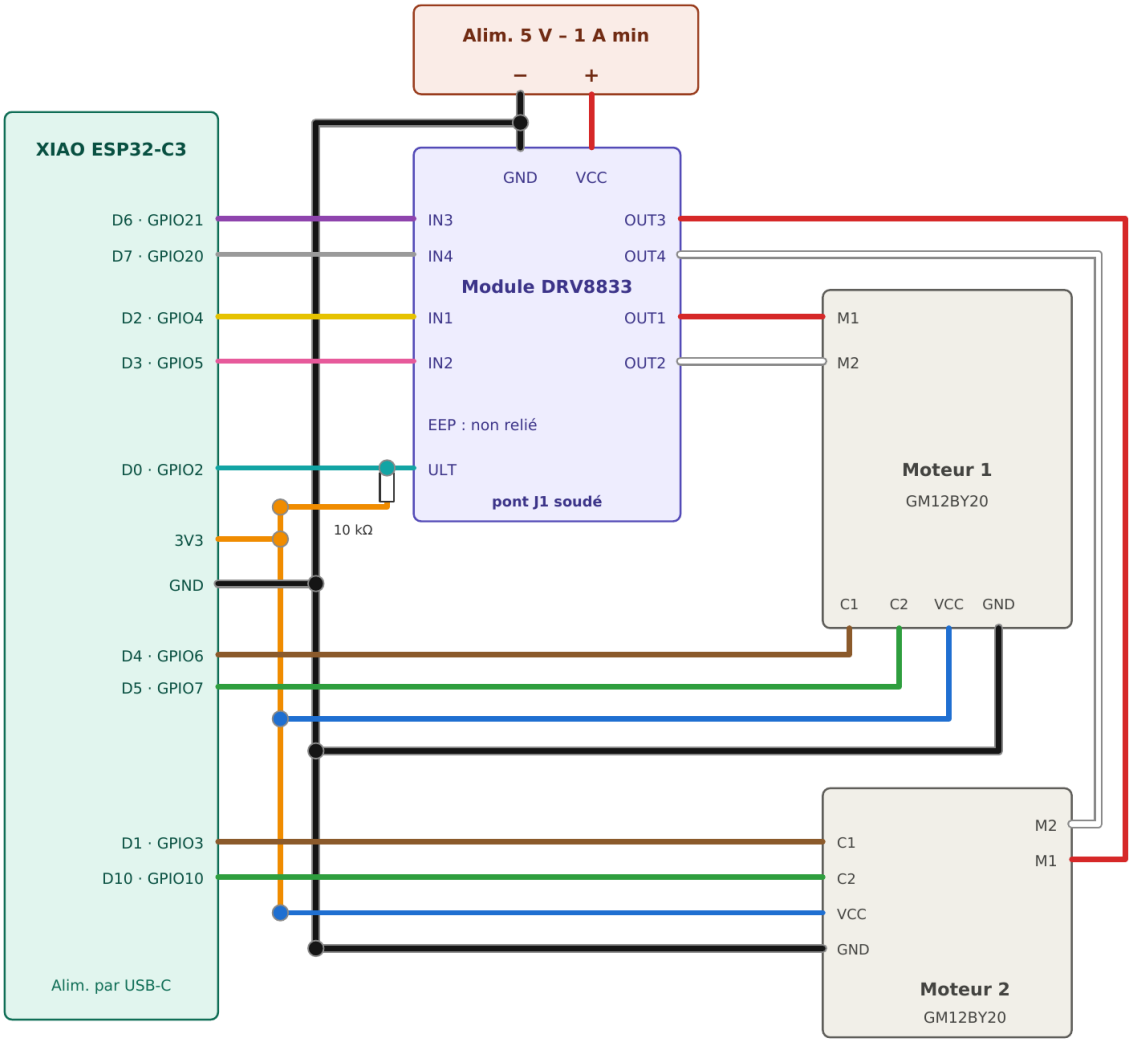
> Variante **puce DRV8833PWPR seule** (sans module) : il faut alors ajouter ses composants externes (10 µF + 100 µF sur VM, 2,2 µF sur VINT, 10 nF entre VCP et VM, 10 kΩ sur nSLEEP et nFAULT, pastille thermique à la masse, éventuellement 0,2 Ω sur AISEN/BISEN). Ils sont détaillés dans la fiche de câblage dédiée à la puce seule.

Les moteurs sont prévus pour 6 V ; alimentés en 5 V, ils tournent environ 15 % moins vite (vers 60 tr/min au lieu de 70) et un peu moins fort. C'est généralement suffisant pour un store et cela ménage le réducteur.



Module DRV8833 1,5A 2-canaux H pont DC

3. Montage et câblage



Un point = connexion · un croisement sans point = pas de contact

Rouge : +5 V, fil M1 du moteur
Blanc : fil M2 du moteur
Bleu : VCC du codeur (3,3 V)
Rose : D3 → IN2
Noir : masse (GND)
Marron : codeur C1 (voie A)
Turquoise : ULT → D0
Violet : D6 → IN3
Orange : +3,3 V
Vert : codeur C2 (voie B)
Jaune : D2 → IN1
Gris : D7 → IN4

Les couleurs entre le XIAO et le module sont des fils Dupont au choix : seul compte de garder le même code partout.
Le jaune n'existe pas sur la nappe du moteur : il est utilisé ici pour la liaison D2 → IN1.

3.1 Liaisons du XIAO

XIAO	GPIO	Relié à	Fonction
D2	4	Module IN1	Moteur 1 — entrée 1 (PWM)
D3	5	Module IN2	Moteur 1 — entrée 2 (PWM)
D6	21	Module IN3	Moteur 2 — entrée 1 (PWM)
D7	20	Module IN4	Moteur 2 — entrée 2 (PWM)
D4	6	Encodeur 1, C1 (fil marron)	Voie A moteur 1

XIAO	GPIO	Relié à	Fonction
D5	7	Encodeur 1, C2 (fil vert)	Voie B moteur 1
D1	3	Encodeur 2, C1 (fil marron)	Voie A moteur 2
D10	10	Encodeur 2, C2 (fil vert)	Voie B moteur 2
D0	2	Module ULT (après vérification, voir 3.2)	Défaut du pont en H (entrée, 0 = défaut)
3V3	—	VCC des encodeurs (fils bleus) ; résistance 10 kΩ de ULT	Alimentation logique
GND	—	Masse commune (alimentation, module, fils noirs des encodeurs)	—
USB-C	—	Alimentation du XIAO	Voir 3.5 pour une alimentation par la broche 5V
D8, D9	8, 9	libres	Broches de démarrage — voir 3.5

3.2 Liaisons du module DRV8833

Repères imprimés sur le module : côté gauche, de haut en bas, **IN4, IN3, GND, VCC, IN2, IN1** ; côté droit, **EEP, OUT1, OUT2, OUT3, OUT4, ULT**.

Repère	Relier à	Repère	Relier à
IN1	XIAO D2 (fil jaune)	OUT1	Moteur 1, fil rouge (M1)
IN2	XIAO D3 (fil rose)	OUT2	Moteur 1, fil blanc (M2)
IN3	XIAO D6 (fil violet)	OUT3	Moteur 2, fil rouge (M1)
IN4	XIAO D7 (fil gris)	OUT4	Moteur 2, fil blanc (M2)
VCC	+5 V de l'alimentation (1 A min.)	EEP	**Non relié** (pont J1 soudé)
GND	− de l'alimentation et GND du XIAO	ULT	XIAO D0 + 10 kΩ vers 3,3 V (voir ci-dessous)

Pont J1. Souder J1 met la puce en marche en permanence : EEP (nSLEEP) est alors relié à VCC. Laisser EEP **non branché** et surtout ne jamais le relier au 3V3 du XIAO : cela mettrait le 5 V et le 3,3 V en contact. Sans J1 soudé, EEP doit être relié au 3,3 V, sinon la puce reste en sommeil et les moteurs ne bougent pas.

ULT : mesurer avant de brancher sur D0. Sur certains modules, ULT est tirée vers VCC (5 V) par une résistance de la carte, or D0 ne supporte pas le 5 V. Alimenter le module en 5 V **sans le XIAO** et mesurer ULT par rapport à GND :
• **0 V ou valeur instable** : pas de rappel sur la carte → relier ULT à D0 et ajouter la 10 kΩ entre ULT et 3,3 V ;
• **environ 5 V** : ne **pas** relier ULT à D0 ; le laisser libre et mettre `NFAULT_ENABLED = false` dans le programme. La puce garde sa protection interne, mais la page web n'affichera plus le défaut.

3.3 Câble moteur 6 fils

Fil	Rôle	Moteur 1 → / Moteur 2 →
Rouge	M1 (puissance)	OUT1 / OUT3
Blanc	M2 (puissance)	OUT2 / OUT4
Marron	C1, voie A de l'encodeur	XIAO D4 / XIAO D1
Vert	C2, voie B de l'encodeur	XIAO D5 / XIAO D10
Bleu	VCC de l'encodeur	XIAO 3V3 (jamais 5 V)
Noir	GND de l'encodeur	Masse commune

Vérification à l'ohmmètre avant de brancher : seule la paire rouge–blanc doit présenter une faible résistance (quelques ohms à quelques dizaines d'ohms : c'est le bobinage du moteur). Si une autre paire réagit ainsi, ne rien brancher : la sérigraphie de la petite carte verte de l'encodeur (M1, GND, C1, C2, 3V3/VCC, M2) fait foi. Les encodeurs ne passent pas par le DRV8833.

3.4 Limitation de courant

Sur le module, les broches de mesure AISEN et BISEN de la puce sont reliées directement à la masse : il n'y a **pas de limitation de courant réglable**. Le DRV8833 garde sa protection interne contre les surintensités et la surchauffe (signalée par ULT), et la détection de blocage du programme (§ 7.6) arrête le moteur en 0,4 s.

3.5 Précautions

Encodeurs en 3,3 V uniquement. Les entrées de l'ESP32-C3 ne tolèrent pas le 5 V : un encodeur alimenté en 5 V renverrait des signaux à 5 V sur D4/D5/D1/D10 et pourrait détruire la carte. Il en va de même pour ULT si le module la tire au 5 V (voir 3.2).

Broches de démarrage. D8 (GPIO8) et D9 (GPIO9) déterminent le mode de démarrage ; D0 (GPIO2) aussi dans une moindre mesure. Un encodeur branché dessus pourrait les tenir à l'état bas au démarrage et bloquer la carte en mode téléchargement. Ne pas y brancher d'encodeur. ULT convient sur D0 : elle est tenue à l'état haut par sa résistance de rappel et ne passe à l'état bas qu'en cas de défaut du DRV8833. Si D0 n'est pas câblé, mettre `NFAULT_ENABLED = false` (par sécurité, l'entrée a de toute façon un rappel interne).

- **Masses communes** : la masse de l'alimentation, celle du module DRV8833 et celle du XIAO doivent être reliées.
- D6/D7 sont normalement le port série matériel du XIAO : ils servent ici au moteur 2, d'où l'obligation d'utiliser le port série par l'USB (§ 4.3).
- **USB et alimentation externe simultanées** : le XIAO est normalement alimenté par son USB-C. Pour l'alimenter par sa broche 5V depuis la même source que le module, débrancher l'USB pendant ce temps (ou intercaler une diode Schottky), afin qu'aucun courant ne reflue vers l'ordinateur.
- Facultatif mais conseillé : un condensateur céramique de **100 nF** directement aux bornes de chaque moteur réduit les parasites.
- Câbles des encodeurs : courts, éloignés des fils moteurs, idéalement torsadés avec la masse.

4. Installation du programme

4.1 Prérequis

- Arduino IDE 2.x.
- Paquet **esp32 d'Espressif** en version 2.x ou 3.x (Gestionnaire de cartes). Le programme s'adapte automatiquement : la fonction PWM a changé en 3.x.
- Aucune bibliothèque supplémentaire : WiFi, WebServer, ESPmDNS, Preferences, ArduinoOTA et Update sont fournies avec le paquet.

4.2 Fichiers du croquis

Le dossier du croquis doit contenir trois fichiers (ils apparaissent comme trois onglets dans l'IDE) :

Fichier	Contenu
`StoreEnrouleur_MoteurCC_Encodeur.ino`	Le programme. Le dossier doit porter le même nom que ce fichier.
`arduino_secrets.h`	Noms de réseau et mots de passe (à créer, voir ci-dessous)
`favicon.h` **(v5.2)**	Icône de la page web : image PNG 180 × 180 px convertie en tableau d'octets, stockée en mémoire flash (environ 2 ko)

Créer `arduino_secrets.h` dans le dossier du croquis :

```
#define SECRET_SSID      "NomDeLaBox"        // réseau Wi-Fi de la maison
#define SECRET_PASS      "MotDePasseWiFi"
#define SECRET_AP_SSID   "Stores-Secours"    // réseau créé par la carte en secours
#define SECRET_AP_PASS   "8caracteresMini"   // 8 caractères minimum
#define SECRET_OTA_PASS  "MotDePasseOTA"     // mises à jour (IDE et /update)
```

> Si `SECRET_OTA_PASS` est absent, le mot de passe par défaut `store-ota` est utilisé : à remplacer. Un mot de passe de point d'accès de moins de 8 caractères empêche la création du réseau de secours.

> **Plusieurs cartes ?** Chaque carte doit avoir un `SECRET_AP_SSID` **différent** (ex. « Store-Salon », « Store-Chambre ») et un `HOSTNAME` différent dans le croquis (§ 8.4 et § 9.1). Les mots de passe peuvent rester identiques.

> **Changer l'icône.** Pour une autre image, la convertir en PNG carré de 180 × 180 px (moins de quelques ko), puis régénérer `favicon.h` en gardant les noms `favicon_png` et `favicon_png_len`. Le programme n'a pas à être modifié.

4.3 Réglages de l'IDE

Menu	Valeur
Outils › Type de carte	**XIAO_ESP32C3**
Outils › USB CDC On Boot	**Enabled** — obligatoire pour voir le moniteur série (D6/D7 sont occupés)
Outils › Partition Scheme	Un schéma avec OTA (le schéma par défaut convient)

Menu	Valeur
Moniteur série	115 200 bauds

4.4 Premier téléversement

1. Adapter si besoin les réglages en tête du croquis (noms des stores, un ou deux moteurs, `NFAULT_ENABLED` selon le câblage de ULT, `HOSTNAME` s'il y a plusieurs cartes : § 8).
2. Brancher la carte en USB, choisir son port, téléverser.
3. Ouvrir le moniteur série : la carte affiche sa bannière, les valeurs relues en mémoire, puis l'adresse IP obtenue.
4. Ouvrir `http://store.local` ou l'adresse IP dans un navigateur.

Les téléversements suivants peuvent se faire sans fil (§ 10).

5. Première mise en service et calibration

Au premier démarrage, aucun store n'est calibré : la page affiche « non calibré » et refuse ▲, ▼ et « Aller ». La calibration se fait **pour chaque moteur séparément**, depuis sa carte « Calibration » (cliquer sur le titre pour l'ouvrir).

> Les calibrations des versions à moteurs pas à pas (v4 et antérieures) ne sont **pas reprises** : la v5 utilise un autre espace mémoire (`rollerdc`) et compte en impulsions d'encodeur.

5.1 Étape 0 — vérifier le sens du moteur et de l'encodeur

Un moteur à courant continu tourne dans un sens ou dans l'autre selon le branchement de ses deux fils, et l'encodeur peut compter positivement dans l'un ou l'autre sens. Le programme attend : **descente = sens +1** et **position qui augmente en descendant**.

1. Appuyer sur **Démarrer** (mode calibration : pas de butée, vitesse réduite).
2. Appuyer brièvement sur ▼, puis ■. Observer :

Constat	Correction
Le store descend et la position brute augmente	Rien à faire : passer à l'étape 1.
Le store **monte** au lieu de descendre	`MOTx_INVERT = true` (ou inverser les fils rouge et blanc du moteur)
Le store descend mais s'arrête avec « sens incohérent »	`ENCx_INVERT = true` (ou inverser les fils marron et vert, C1 et C2)
Le store monte **et** s'arrête sur « sens incohérent »	Seul le moteur est à l'envers : `MOTx_INVERT = true` , puis refaire le test
« BLOCAGE détecté » sans mouvement	Voir § 13 (encodeur non câblé, `SPEED_MIN` trop faible, J1 non soudé…)

Après modification du programme, re-téléverser et refaire l'étape 0.

5.2 Étape 1 — position haute

1. Avec ▲, ▼ et ■, amener le store exactement à la position haute souhaitée.

2. Moteur arrêté, appuyer sur **Valider HAUT** : la position brute passe à 0.

5.3 Étape 2 — position basse

1. Descendre le store exactement à la position basse souhaitée, puis ■.
2. Appuyer sur **Valider BAS** : la position courante devient la course ; elle est enregistrée et les butées sont réactivées. La position affiche 100 %.

5.4 Règles et conseils

- « Valider HAUT » et « Valider BAS » exigent un moteur **à l'arrêt**. « Valider BAS » exige que le haut ait été validé et que la position soit en dessous du haut.
- « Valider HAUT » peut être refait autant de fois que nécessaire avant « Valider BAS ».
- En calibration, le moteur s'arrête seul en cas de blocage (0,4 s contre une butée mécanique). Valider à cet endroit reste possible, mais mieux vaut s'arrêter juste avant pour ne pas forcer la toile à chaque cycle.
- Une calibration commencée mais non terminée n'est **pas enregistrée** ; en cas de redémarrage, l'ancienne calibration revient. Si le store a bougé pendant ce temps, refaire la calibration jusqu'au bout.
- Pour recalibrer un store déjà calibré : recommencer simplement par « Démarrer ».

6. Utilisation de la page web

L'utilisation quotidienne est décrite en détail dans la **notice d'utilisation de la page web**. En résumé :

Élément	Rôle
Carte « Connexion »	OK / Mise à jour en cours / Hors ligne ; signal Wi-Fi en dBm
Carte « Les deux Stores »	▲ ■ ▼ et « Aller (les deux) » — visible avec deux moteurs seulement
Carte d'un store	Position en %, barre, état, ▲ ■ ▼, curseur « Aller à » et bouton « Aller »
Carte « Calibration »	Démarrer, Valider HAUT, Valider BAS ; position brute et course en impulsions
Message orange	Raison d'un refus de commande
Lien « Mise à jour »	Page `/update`
Icône **(v5.2)**	Affichée dans l'onglet du navigateur et sur l'écran d'accueil du téléphone

La page interroge la carte toutes les 500 ms ; ses cartes sont construites automatiquement à partir de la réponse (une ou deux selon `MOTOR2_ENABLED`).

6.1 Raccourci sur l'écran d'accueil (v5.2)

Ouvrir la page, puis dans le menu du navigateur choisir **« Ajouter à l'écran d'accueil »** (Chrome sous Android, Safari sous iPhone via le bouton Partager). Le raccourci prend l'icône du store. Avec plusieurs cartes, renommer chaque raccourci (ex. « Store salon »). Le navigateur garde l'icône en mémoire 7 jours : après un changement d'icône, vider son cache ou supprimer puis recréer le raccourci.

7. Fonctionnement interne

7.1 Organisation du programme

Partie	Rôle	Rythme
Interruptions encodeurs (`encoderISR`)	Comptent chaque front des voies A et B	À chaque front
Temporisateur `esp_timer` (`controlTick`)	Régule les moteurs : rampe, ralentissement, butées, sécurités	Toutes les 10 ms
Boucle `loop()`	Serveur web, OTA, sauvegardes en mémoire, messages série	En continu

La régulation tourne dans le temporisateur, indépendamment du Wi-Fi : une page web lente ne retarde jamais un arrêt en butée. Chaque moteur possède sa propre structure de données (`Motor`) : les deux stores sont réellement indépendants.

7.2 Encodeur en quadrature ×4

Les voies A et B sont décalées d'un quart de période. À chaque front de A ou de B, le programme lit les deux voies d'un seul coup dans le registre des GPIO et compare l'ancien et le nouvel état dans une table à 16 cases : le résultat est +1, −1 ou 0 (rebond ignoré). On obtient ainsi **4 impulsions par période** de l'encodeur et le sens de rotation.

Nombre d'impulsions par tour d'arbre de sortie = 4 × (impulsions par tour de l'encodeur) × (rapport du réducteur). Exemple : encodeur 7 impulsions/tour, réducteur 1:150 → 4 200 impulsions par tour de sortie. La course d'un store se compte donc couramment en dizaines de milliers d'impulsions : la résolution est très fine. Même à pleine vitesse, la fréquence des fronts (quelques kHz) reste très faible pour l'ESP32-C3.

7.3 Commande PWM du DRV8833

Chaque moteur reçoit deux signaux PWM à **20 kHz** (au-delà de l'audible), en résolution 10 bits (0 à 1023), sur les entrées IN1/IN2 (moteur 1) ou IN3/IN4 (moteur 2) du module. Le programme utilise le mode « **décroissance lente** » : une entrée reste à 1, l'autre reçoit le PWM inversé. Pendant les temps morts, le courant circule en roue libre par les transistors bas, ce qui donne une vitesse presque proportionnelle au rapport cyclique, même à basse vitesse.

Situation	IN1 (IN3)	IN2 (IN4)	Effet
Descente, vitesse *v*	1023	1023 − *v*	Rotation sens +
Montée, vitesse *v*	1023 − *v*	1023	Rotation sens −
Arrêt	1023	1023	Frein : bobinage court-circuité

`MOTx_INVERT` échange simplement les deux entrées par logiciel.

7.4 Rampe, ralentissement et arrêt

- **Démarrage** : le moteur part directement à `SPEED_MIN` (400), puis la vitesse augmente de `RAMP_STEP` (40) toutes les 10 ms jusqu'à la vitesse souhaitée : environ 0,16 s pour atteindre 1023.
- **Vitesse souhaitée** : `SPEED_MAX` (1023) en service normal, `SPEED_CAL` (600) en calibration.
- **Ralentissement** : à l'approche d'un point d'arrêt (cible « Aller à » ou butée), dans une zone égale à `SLOW_ZONE_PCT` (5 %) de la course, au minimum `SLOW_ZONE_MIN` (40) impulsions, la vitesse diminue

proportionnellement à la distance restante :
`vitesse = SPEED_MIN + (souhaitée − SPEED_MIN) × distance / zone`.
Exemple : course de 20 000 impulsions → zone de 1 000 impulsions ; à 500 impulsions de la cible, vitesse ≈ 400 + 623 × 0,5 ≈ 711.

- **Arrêt** : dès que la position atteint ou dépasse le point d'arrêt, le moteur est freiné. Grâce au ralentissement, le dépassement est de quelques impulsions seulement.
- En calibration, il n'y a ni butée ni ralentissement : le moteur tourne à `SPEED_CAL` jusqu'à Stop.

7.5 Inversion de sens et relance

Si une commande demande le sens opposé pendant un mouvement, ou si un moteur est relancé moins de `RESTART_WAIT_MS` (300 ms) après un arrêt, le programme freine et attend `RESTART_WAIT_TICKS` × 10 ms (200 ms) avant de repartir de la vitesse minimale. Cela évite les pointes de courant et les chocs dans le réducteur, et laisse le store finir de glisser avant de prendre la nouvelle position de départ. Une commande dans le même sens que le mouvement en cours conserve la vitesse acquise.

7.6 Sécurités

Sécurité	Condition	Réaction
Blocage	Moteur alimenté mais aucune impulsion d'encodeur pendant `STALL_MS` (400 ms) : store coincé, butée mécanique, moteur trop faible, encodeur débranché	Arrêt freiné, état « BLOCAGE détecté »
Sens incohérent	La position recule de plus de `WRONG_DIR_COUNTS` (60) impulsions par rapport au sens commandé	Arrêt freiné, état « sens incohérent »
Butées logicielles	Position ≤ 0 en montée, ≥ course en descente	Arrêt freiné
Non calibré	Course = 0 et calibration non démarrée	▲ ▼ « Aller » refusés
Défaut DRV8833	ULT (nFAULT) à l'état bas pendant 30 ms (`NFAULT_TICKS` × 10 ms) : surintensité (court-circuit) ou surchauffe du circuit	Arrêt freiné des deux moteurs, état « DÉFAUT DRV8833 » ; mouvements refusés tant que ULT reste bas
Mise à jour	Écriture d'un nouveau programme	Moteurs arrêtés, commandes refusées

Un défaut reste affiché jusqu'à la commande suivante (Stop compris), qui l'efface. Le DRV8833 ne permet pas de savoir quel pont est en cause : c'est pourquoi les deux moteurs sont arrêtés. Seules la protection interne contre les surintensités et l'arrêt thermique abaissent ULT. Si ULT n'est pas reliée à D0 (`NFAULT_ENABLED = false`), ces protections agissent toujours, mais sans message sur la page.

7.7 Sauvegarde de la position

Donnée	Clé NVS (espace `rollerdc`)	Quand est-elle écrite ?
Course du moteur 1 / 2	`travel1` / `travel2`	À « Valider BAS »
Position du moteur 1 / 2	`pos1` / `pos2`	Pendant un mouvement : au plus toutes les 1,5 s ; à l'arrêt : 300 ms après, une fois le glissement terminé ; avant une mise à jour

Une position identique à la dernière enregistrée n'est jamais réécrite, ce qui ménage la mémoire flash. Aucune position n'est enregistrée pendant une calibration. Au démarrage, les valeurs relues sont bornées entre 0 et la course.

> Après une coupure de courant en plein mouvement, la position mémorisée peut avoir jusqu'à 1,5 s de retard sur la réalité, et l'encodeur ne compte pas si le store tourne alors qu'il n'est pas alimenté. En cas d'écart visible, refaire une calibration.

7.8 Icône de la page (v5.2)

`favicon.h` contient l'image PNG sous forme de tableau `favicon_png` placé en mémoire flash (`PROGMEM`) ; aucune mémoire vive n'est utilisée. La fonction `handleFavicon()` l'envoie telle quelle avec le type `image/png` et l'en-tête `Cache-Control: max-age=604800` : le navigateur la garde 7 jours et ne la redemande pas à chaque rafraîchissement. Les pages `/` et `/update` la déclarent par des balises `<link rel="icon">` et `<link rel="apple-touch-icon">`.

8. Réglages du programme

Les constantes marquées 🚩 dans le croquis sont celles que l'on est amené à modifier.

8.1 Moteurs

Constante	Défaut	Rôle
`MOTOR2_ENABLED`	true	false : un seul store ; D6, D7, D1, D10 inutilisés ; page sur une colonne
`MOT1_INVERT` , `MOT2_INVERT`	false	Inverse le sens de rotation
`ENC1_INVERT` , `ENC2_INVERT`	false	Inverse le sens de comptage
`MOT1_NAME` , `MOT2_NAME`	"Store 1", "Store 2"	Noms affichés (ex. « Store salon »). Éviter les guillemets droits dans le nom

8.2 Vitesses (0 à 1023)

Constante	Défaut	Rôle et conseils
`SPEED_MAX`	1023	Vitesse normale. Baisser (ex. 800) pour un store plus lent et plus silencieux.
`SPEED_CAL`	600	Vitesse en calibration, pour viser les butées confortablement.
`SPEED_MIN`	400	Doit suffire à faire démarrer le store **en montée** (cas le plus chargé). Si « BLOCAGE » apparaît au démarrage : augmenter par pas de 50.
`RAMP_STEP`	40	Accélération par période de 10 ms. Plus petit = démarrage plus doux.
`SLOW_ZONE_PCT`	5	Longueur de la zone de ralentissement en % de la course.
`SLOW_ZONE_MIN`	40	Longueur minimale de cette zone, en impulsions.

8.3 Sécurités et temps

Constante	Défaut	Rôle et conseils
`STALL_MS`	400 ms	Délai sans impulsion avant « BLOCAGE ». Plus court = plus réactif mais risque de fausses alertes au démarrage.
`WRONG_DIR_COUNTS`	60	Recul toléré avant « sens incohérent » (un store chargé peut reculer un peu au démarrage).
`RESTART_WAIT_TICKS`	20	Pause freinée avant relance (× 10 ms).
`RESTART_WAIT_MS`	300 ms	En deçà, une relance après arrêt déclenche la pause.
`SETTLE_MS`	300 ms	Attente après arrêt avant sauvegarde définitive.
`SAVE_EVERY_MS`	1500 ms	Intervalle minimal entre deux sauvegardes en mouvement.
`PWM_FREQ`	20 000 Hz	Fréquence PWM (ne pas descendre sous 20 kHz pour rester inaudible).
`NFAULT_ENABLED`	true	Surveillance de ULT (nFAULT) sur D0. **false** si ULT n'est pas reliée à D0, notamment quand le module la tire au 5 V (§ 3.2).
`NFAULT_TICKS`	3	Durée de confirmation d'un défaut (× 10 ms).

8.4 Réseau

Constante	Défaut	Rôle
`HOSTNAME`	"store"	Nom réseau → `http://store.local` et nom dans l'IDE. **Unique pour chaque carte** s'il y en a plusieurs (ex. "store-salon" → `http://store-salon.local`)
`AP_IP`	192.168.5.1	Adresse de la carte en point d'accès de secours. Peut rester la même sur toutes les cartes (§ 9.1)
`OTA_PORT`	3232	Port OTA Arduino
`AP_RETRY_MS`	300 000 ms (5 min)	Intervalle des tentatives de retour sur la box en mode secours
`AP_TRY_MS`	15 000 ms	Durée d'une tentative

9. Réseau Wi-Fi

- Au démarrage, la carte tente de rejoindre la box pendant **15 s**. En cas de succès, le moniteur série affiche l'adresse IP et la force du signal.
- En cas d'échec, elle crée son propre réseau (`SECRET_AP_SSID`) à l'adresse **192.168.5.1**. La page affiche alors « OK — point d'accès de secours ».
- **Retour automatique** : toutes les 5 minutes, si **aucun appareil n'est connecté** au point d'accès, la carte retente la box pendant 15 s, sans interrompre la commande des moteurs. Si la box répond, le point d'accès est coupé et la carte redevient joignable par `store.local` ; sinon le point d'accès reste en place jusqu'à la tentative suivante. Un appareil connecté au réseau de secours reporte la tentative, pour ne jamais couper quelqu'un qui commande les stores.
- Une fois sur la box, une coupure passagère de celle-ci est gérée par la reconnexion automatique de l'ESP32 ; la carte ne repasse pas en mode secours.

- Le nom `store.local` (mDNS) est annoncé dans les deux modes, ainsi que le service OTA pour l'Arduino IDE.
- La veille Wi-Fi est désactivée (réactivité) et la puissance d'émission est réduite à 8,5 dBm, ce qui évite les problèmes de connexion connus sur le XIAO ESP32-C3. Si la carte est loin de la box (signal sous −75 dBm), rapprocher la carte ou ajouter un répéteur.
- Le XIAO ne fonctionne qu'en **2,4 GHz**.

> Conseil : réserver une adresse IP fixe pour la carte dans la box (bail DHCP statique). Les téléphones Android qui ne résolvent pas `.local` pourront ainsi toujours utiliser la même adresse.

9.1 Plusieurs cartes ESP32

Chaque carte en mode secours crée **son propre réseau Wi-Fi, isolé des autres**. Un téléphone n'étant connecté qu'à un seul réseau à la fois, deux cartes à la même adresse 192.168.5.1 ne se gênent jamais : il n'est **pas nécessaire** de changer `AP_IP`. En revanche, sur la box, toutes les cartes partagent le même réseau.

Réglage	Sur chaque carte	Pourquoi
`SECRET_AP_SSID`	**Différent**	Pour savoir à quelle carte on se connecte en mode secours (ex. « Store-Salon », « Store-Chambre »)
`HOSTNAME`	**Différent**	Sur la box, deux cartes nommées `store` se disputeraient `store.local`, et l'OTA de l'IDE pourrait viser la mauvaise carte
`AP_IP`	Identique possible	Réseaux de secours séparés. Des adresses différentes (192.168.5.1, 192.168.6.1…) restent possibles pour s'y retrouver
Mots de passe (AP, OTA)	Identiques possibles	Au choix
`MOT1_NAME`, `MOT2_NAME`	Conseillé différent	La page indique ainsi clairement quels stores on commande

> La plage du réseau de secours (192.168.5.x) ne doit pas être celle de la box : pendant les tentatives de retour, la carte est reliée aux deux réseaux à la fois. Une Livebox utilise habituellement 192.168.1.x ; vérifier l'adresse affichée par le moniteur série lors d'une connexion normale.

9.2 Accéder à une carte en mode secours

En mode secours, on choisit la carte en choisissant son réseau Wi-Fi :

1. Dans les réglages Wi-Fi du téléphone ou du PC, se connecter au réseau de la carte voulue (ex. « Store-Salon ») avec le mot de passe du point d'accès.
2. Ouvrir `http://192.168.5.1` dans le navigateur.
3. Pour passer à l'autre carte : changer de réseau Wi-Fi (« Store-Chambre ») et recharger la même adresse : c'est maintenant l'autre carte qui répond.

- **« Pas de connexion Internet »** : message normal. Sous Android, répondre **« Rester connecté »** ; sinon le téléphone peut repasser seul en 4G/5G et la page ne s'affiche plus. Au besoin, couper temporairement les données mobiles.
- **Utiliser l'adresse IP** plutôt que le nom `.local`, qui ne fonctionne pas sur tous les appareils en mode secours (Android notamment).

- Le téléphone peut revenir seul sur la box s'il la capte : si la page ne répond plus, vérifier le réseau Wi-Fi utilisé.
- Un même appareil ne commande qu'une carte à la fois en mode secours. Pour commander toutes les cartes ensemble, elles doivent être sur la box (ou utiliser plusieurs appareils).

10. Mises à jour (OTA)

10.1 Depuis l'Arduino IDE

1. Carte et ordinateur sur le même réseau.
2. Outils › Port : choisir le port réseau **store** (avec son adresse IP) ; avec plusieurs cartes, le nom choisi dans `HOSTNAME`.
3. Téléverser ; saisir le mot de passe OTA quand l'IDE le demande.

Le moniteur série (s'il est relié en USB) affiche la progression par dizaines de %. Le moniteur série de l'IDE n'est pas disponible par le réseau.

10.2 Depuis le navigateur

1. Arduino IDE : **Croquis › Exporter les binaires compilés**. Le fichier se trouve dans le dossier `build/…` du croquis ; prendre celui qui se termine par `.ino.bin` (pas `.merged.bin` ni `.bootloader.bin`).
2. Ouvrir `http://store.local/update`, saisir le mot de passe OTA, choisir le fichier, **Envoyer**.
3. « Terminé, redémarrage… » : la carte redémarre et la page principale revient au bout de 10 s.

10.3 Comportement pendant une mise à jour

- Les deux moteurs sont arrêtés et freinés, les positions enregistrées ; une calibration en cours est abandonnée.
- La régulation des moteurs est suspendue pour laisser tout le processeur à l'écriture ; les commandes sont refusées (« Mise à jour en cours, patientez »).
- En cas d'échec ou d'interruption, l'ancien programme reste en place et les moteurs redeviennent commandables.
- Sur `/update`, la page vérifie d'abord le mot de passe (`/update/check`) : s'il est faux, « Échec : mot de passe incorrect » s'affiche aussitôt et rien n'est envoyé. Chaque mauvais essai est retardé d'une seconde. La carte vérifie à nouveau le mot de passe à la réception du fichier.
- Un fichier refusé affiche « Échec : fichier invalide ou incomplet ».

> Une mise à jour conserve la calibration et la position (même espace mémoire). En revanche, le passage d'une version à moteurs pas à pas à la v5 impose une nouvelle calibration. Le passage de la v5.1 à la v5.2 ne demande aucune recalibration ; `favicon.h` doit seulement être présent dans le dossier du croquis pour compiler.

11. Interface de programmation (API HTTP)

La page web utilise deux adresses que l'on peut aussi appeler depuis une domotique (Home Assistant, Jeedom, script, raccourci de téléphone…).

11.1 Lire l'état : `GET /api/status`


```
{"ota":false,"ap":false,"drv":false,"rssi":-58,"m":[
 {"name":"Store 1","pct":35,"step":7012,"travel":20034,"dir":1,"fault":0,
  "cal":false,"top":false,"ready":true},
 {"name":"Store 2","pct":100,"step":19870,"travel":19870,"dir":0,"fault":0,
  "cal":false,"top":false,"ready":true}]}
```


Champ	Signification
`ota`	Mise à jour en cours
`ap`	Mode point d'accès de secours
`drv`	Défaut DRV8833 en cours (ULT / nFAULT bas)
`rssi`	Signal de la box en dBm (0 en point d'accès)
`name`	Nom du store
`pct`	Position 0–100 % (0 si non calibré)
`step`	Position brute en impulsions
`travel`	Course totale en impulsions (0 = non calibré)
`dir`	1 = descente, −1 = montée, 0 = arrêt
`fault`	0 = aucun, 1 = blocage, 2 = sens incohérent, 3 = défaut DRV8833
`cal` / `top`	Calibration en cours / haut validé
`ready`	Store calibré

11.2 Commander : `GET /api/cmd?a=<action>&m=<moteur>[&p=<%>]`

Paramètre	Valeurs
`m`	`1` ou `2` : un store ; `0` : les deux
`a`	`up` , `down` , `stop` , `goto` (avec `p` = 0 à 100), `calstart` , `caltop` , `calbot` (calibration : `m` = 1 ou 2 obligatoire)
`p`	Pourcentage pour `goto` , borné entre 0 et 100

Réponse : code **200** si au moins un moteur a accepté (texte vide, ou raison du refus de l'autre) ; code **409** si tout est refusé (texte = raison, préfixée du nom du store).

```
http://store.local/api/cmd?a=goto&m=0&p=30     les deux stores à 30 %
http://store.local/api/cmd?a=down&m=1          descendre le store 1
http://store.local/api/cmd?a=stop&m=0          tout arrêter
curl "http://192.168.1.42/api/cmd?a=up&m=2"   depuis un terminal ou un script
```

Avec plusieurs cartes, chaque carte a sa propre API : remplacer `store.local` par le nom (`HOSTNAME`) ou l'adresse IP de la carte visée.

11.3 Autres adresses

Adresse	Usage
`/`	Page de commande
`/update` , `/update/check`	Mise à jour par le navigateur (§ 10.2)
`/favicon.ico` , `/favicon.png` , `/apple-touch-icon.png` **(v5.2)**	Icône de la page (même image PNG pour les trois)

> L'API et la page de commande **n'ont pas de mot de passe**. Toute personne connectée au réseau local peut commander les stores. Ne pas ouvrir de port vers Internet sur la box ; pour un accès à distance, passer par un VPN.

12. Messages du moniteur série

Message	Signification
`=== XIAO ESP32-C3 Stores enrouleurs (web) v5.2 … ===`	Démarrage
`[NVS] Store 1 : course = N impulsions, position = P`	Valeurs relues en mémoire
`[WIFI] Connexion.....`	Tentative de connexion à la box
`[WIFI] CONNECTÉ, IP : …, signal : -xx dBm`	Connecté : noter l'adresse
`[WIFI] ÉCHEC → point d'accès de secours` `[WIFI] Réseau « … », IP : 192.168.5.1`	Box introuvable, réseau de secours créé
`[WIFI] Page web : http://store.local`	mDNS actif (avec le nom choisi dans `HOSTNAME`)
`[WIFI] Nouvelle tentative de connexion à la box`	Mode secours : essai périodique
`[WIFI] Retour sur la box, IP : …`	Box retrouvée, point d'accès coupé
`[WIFI] Box toujours introuvable…` / `Point d'accès utilisé : tentative … reportée`	Échec ou report de la tentative
`[OTA] Prêt (Arduino IDE)` / `[WEB] Serveur démarré`	Services prêts
`[WEB] Commande : up, moteur 1`	Commande reçue
`[MOTEUR] Store 1 : ARRÊT à N impulsions (P %)`	Arrêt, suivi éventuellement de `— BLOCAGE` , `— SENS INCOHÉRENT` ou `— DÉFAUT DRV8833`
`[CAL] Store 1 : DÉMARRÉE / HAUT validé / TERMINÉE`	Étapes de calibration
`[NVS] Store 1 : course enregistrée = N impulsions`	Calibration enregistrée
`[OTA] Début…` , `[OTA] 40 %` , `[OTA] Terminée` , `[OTA] Erreur n`	Mise à jour depuis l'IDE
`[OTA-WEB] Fichier : …` , `Reçu : N octets` , `Envoi interrompu`	Mise à jour par le navigateur

13. Dépannage

Symptôme	Causes possibles	Solutions
Rien sur le moniteur série	USB CDC On Boot désactivé ; mauvais port	Outils › USB CDC On Boot › Enabled, re-téléverser
Erreur de compilation « favicon.h: No such file » **(v5.2)**	`favicon.h` absent du dossier du croquis	Copier `favicon.h` à côté du `.ino`, puis rouvrir le croquis
La carte reste en mode téléchargement au démarrage	Quelque chose tire D8/D9 (ou D0) à la masse	Libérer ces broches
`store.local` introuvable	Appareil Android, réseau invité, mDNS filtré	Utiliser l'adresse IP ; réserver une IP fixe
`store.local` ouvre parfois l'autre carte ; l'IDE téléverse sur la mauvaise carte	Plusieurs cartes avec le même `HOSTNAME`	Donner un `HOSTNAME` différent à chaque carte (§ 9.1)
Page « Hors ligne » par intermittence	Signal faible, alimentation instable	Vérifier le dBm ; alimentation 1 A ; masses communes
Carte toujours en mode secours	SSID/mot de passe faux, box en 5 GHz seulement, un appareil reste connecté au point d'accès	Corriger `arduino_secrets.h` ; activer le 2,4 GHz ; déconnecter les appareils du réseau de secours (ou redémarrer la carte)
En mode secours, 192.168.5.1 ne répond pas	Téléphone repassé en 4G/5G ou revenu sur la box ; connecté au réseau d'une autre carte	« Rester connecté » au réseau sans Internet ; couper les données mobiles ; vérifier le nom du réseau Wi-Fi (§ 9.2)
Ancienne icône, ou pas d'icône **(v5.2)**	Icône gardée en cache par le navigateur (7 jours)	Vider le cache ; supprimer puis recréer le raccourci de l'écran d'accueil
« BLOCAGE » dès le démarrage, sans mouvement	Moteur ne reçoit rien : pont J1 non soudé (et EEP non relié), VCC du module absent ; `SPEED_MIN` trop faible	Souder J1 ; vérifier 5 V entre VCC et GND du module ; vérifier les fils OUT ; augmenter `SPEED_MIN`
« BLOCAGE » alors que le moteur tourne	Encodeur non alimenté, fils marron/vert (C1/C2) débranchés	Vérifier VCC encodeur (3,3 V, fil bleu), masse, C1/C2 ; regarder la position brute en calibration
« BLOCAGE » en montée seulement	Store trop lourd/serré	Augmenter `SPEED_MIN` ; vérifier la mécanique ; envisager un réducteur plus démultiplié
« sens incohérent »	Encodeur compte à l'envers	`ENCx_INVERT = true` (ou inverser marron et vert)
▼ fait monter le store	Fils moteur inversés	`MOTx_INVERT = true` (ou inverser rouge et blanc)
Position qui dérive au fil des jours	Glissement de la toile sur le tube, déplacements hors tension	Recalibrer ; vérifier l'accouplement moteur/tube
Position brute qui saute ou compte trop	Parasites sur les fils d'encodeur	Raccourcir/éloigner les câbles, 100 nF aux bornes du moteur

Symptôme	Causes possibles	Solutions
Le store s'arrête un peu après la cible	Inertie	Augmenter `SLOW_ZONE_PCT` ou baisser `SPEED_MIN` (si le démarrage le permet)
Sifflement du moteur	PWM audible (fréquence modifiée)	Remettre `PWM_FREQ` à 20 000
« DÉFAUT DRV8833 »	Court-circuit d'un moteur ou d'une sortie OUT, surchauffe du module, courant élevé	Vérifier les fils moteurs (paire rouge–blanc à l'ohmmètre) ; laisser refroidir ; vérifier la mécanique
« DÉFAUT DRV8833 » permanent dès la mise sous tension	D0 tiré à la masse par autre chose ; ULT sans résistance de rappel	Vérifier le câblage de D0 et la 10 kΩ de ULT ; sinon `NFAULT_ENABLED = false`
Rien ne fonctionne après avoir relié EEP au 3V3	J1 soudé : 5 V et 3,3 V en contact	Débrancher immédiatement EEP ; avec J1 soudé, EEP reste libre
Page `/update` : « Échec : fichier invalide ou incomplet »	Mauvais fichier (`merged` , `bootloader`), schéma de partition sans OTA	Prendre le `.ino.bin` ; partition avec OTA
Le port réseau « store » n'apparaît pas dans l'IDE	mDNS bloqué, pare-feu	Redémarrer l'IDE ; utiliser `/update` ; autoriser le port 3232
« non calibré » après une mise à jour depuis la v4	Nouvel espace mémoire en v5	Normal : calibrer chaque store

14. Sécurité, limites et entretien

- **Pas de fin de course physique** : les butées sont logicielles. Elles reposent sur le comptage de l'encodeur ; la détection de blocage sert de filet de sécurité. Vérifier de temps en temps que le haut et le bas restent justes.
- **Pas de limitation de courant réglable** sur le module : la protection interne du DRV8833 et la détection de blocage (0,4 s) protègent moteur et alimentation.
- **Pas d'authentification** sur la page de commande et l'API : réserver l'accès au réseau local.
- En mode secours, la tentative de retour sur la box n'a lieu que si personne n'est connecté au point d'accès.
- Les écritures en mémoire flash sont limitées au strict nécessaire ; la NVS de l'ESP32 répartit l'usure : aucune intervention n'est nécessaire.
- Entretien : contrôler les vis d'accouplement moteur/tube et l'état de la toile ; recalibrer après toute intervention mécanique.

15. Annexes

15.1 Correspondance broches XIAO ESP32-C3

XIAO	D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
GPIO	2	3	4	5	6	7	21	20	8	9	10
Usage	ULT*	Enc2 A	IN1	IN2	Enc1 A	Enc1 B	IN3	IN4	libre*	libre*	Enc2 B

* broches de démarrage : ne rien y brancher qui les force à l'état bas au démarrage (ULT, tenue à l'état haut par sa résistance de rappel, convient sur D0).

15.2 Historique des versions

Version	Évolution
v2	Sauvegarde de la position pendant le mouvement, mise à jour OTA
v3	Deuxième moteur sur 4 GPIO séparés, alignement des deux côtés
v3.1	OTA réparée, page `/update`, force du signal Wi-Fi
v4	Les deux moteurs deviennent indépendants
v4.1 / v4.2	Mise en page 2 colonnes, boutons ▲ ■ ▼
v5	Motoréducteurs CC à encodeur magnétique + DRV8833 : position par encodeur ×4, PWM 20 kHz, rampe, ralentissement, frein actif, détection de blocage et de sens incohérent, pause avant inversion. Nouvel espace mémoire : recalibration obligatoire.
v5.1	Page `/update` : mot de passe vérifié avant l'envoi du fichier et message d'échec simplifié ; mode secours : retour automatique sur la box toutes les 5 min ; lecture de nFAULT (ULT) sur D0 (défaut DRV8833). Calibration et position de la v5 conservées.
Manuel révisé	Câblage avec module DRV8833 (IN1–IN4, OUT1–OUT4, EEP, ULT, pont J1) ; nappe moteur sans fil jaune (C1 = marron) ; vérification de ULT avant branchement. Programme inchangé.
v5.2	Icône de la page web (fichier `favicon.h`) : onglet du navigateur et écran d'accueil du téléphone, adresses `/favicon.ico`, `/favicon.png`, `/apple-touch-icon.png`. Manuel : plusieurs cartes ESP32 (§ 9.1) et accès en mode secours (§ 9.2). Calibration et position conservées.